7zpus.swe@gmail.com

Specifica Tecnica

Tabella di Versionamento

Versione	Data	Autore	Verificatore	Descrizione
0.1.0	2026/02/28	Laoud Zakaria	Georgescu Diana	Creazione del documento e stesura struttura iniziale

Indice

1 Introduzione **5**
1.1 Obiettivo del documento . . . 5
1.2 Glossario . . . 5
1.3 Riferimenti . . . 5
1.3.1 Riferimenti normativi . . . 5
1.3.2 Riferimenti informativi . . . 5

2 Tecnologie utilizzate **7**

3 Linguaggio di Programmazione: TypeScript **7**
3.1 Deployment: Electron . . . 7
3.2 Framework Frontend: Angular . . . 8
3.3 Librerie Aggiuntive . . . 9
3.4 Tecnologie di analisi statica . . . 10
3.5 Tecnologie di analisi dinamica . . . 10

4 Contesto del sistema **10**
4.1 Modello architetturale . . . 11
4.2 Descrizione degli attori esterni . . . 11

5 Architettura dei container **12**
5.1 Renderer . . . 13
5.1.1 Struttura architetturale . . . 13
5.2 [Main] . . . 13
5.2.1 Tecnologia: Node.js . . . 13

6 Architettura delle componenti **14**
6.1 [Componente 1] . . . 14
6.1.1 Descrizione . . . 14
6.1.2 Interfacce . . . 14
6.1.3 Dipendenze . . . 14
6.1.4 Tracciamento dei requisiti . . . 14
6.2 [Componente 2] . . . 14
6.2.1 Descrizione . . . 14
6.2.2 Interfacce . . . 14
6.2.3 Dipendenze . . . 14
6.2.4 Tracciamento dei requisiti . . . 14
6.3 Componenti di Verifica Integrità (Frontend) . . . 15
6.3.1 Descrizione . . . 15
6.3.2 Componente di verifica . . . 15
6.3.3 Componenti di presentazione . . . 16
6.4 Componenti di Ricerca (FrontEnd) . . . 17
6.4.1 Descrizione . . . 17

6.4.2 Componenti di presentazione 18

Elenco delle figure

1 Diagramma C1 - Contesto del sistema *DIPReader* 11
2 Diagramma C2 - Container del sistema *DIPReader* 12
3 Diagramma delle classi Verifica Integrità (Frontend) 17

Elenco delle tabelle

2 Dettagli sulla scelta di TypeScript come linguaggio di programmazione . 7
4 Dettagli sulla scelta di Electron come piattaforma di deployment 7
6 Dettagli sulla scelta di Angular come framework frontend 8
8 Dettagli sulle scelte di librerie aggiuntive 9
10 Dettagli sulle scelte di tecnologie di analisi statica 10
12 Dettagli sulle scelte di tecnologie di analisi dinamica 10

1 Introduzione

1.1 Obiettivo del documento

L'obiettivo del presente documento è descrivere in modo rigoroso le scelte tecnologiche e l'architettura del prodotto *DIPReader*, adottando il modello di progettazione *C4*.
L'analisi verrà condotta partendo dalla descrizione del contesto applicativo, per poi approfondire la struttura dei container, delle componenti e, infine, del codice, al fine di fornire una rappresentazione completa, coerente e sistematica del sistema.

1.2 Glossario

Ogni termine tecnico o con un significato particolare nell'ambito dell'Ingegneria del Software, utilizzato nella documentazione di progetto, è definito nell'apposito documento Glossario (v2.0.0).

1.3 Riferimenti

1.3.1 Riferimenti normativi

- Norme di Progetto v2.0.0 (v2.0.0)
- Standard SDD/IEEE/1016:2009(ultimo accesso: 02/03/2026)
- Regolamento di Progetto Didattico a.a. 2025/2026 (ultimo accesso: 02/03/2026)

1.3.2 Riferimenti informativi

- Capitolato C3: DIPReader (ultimo accesso: 10/02/2026)
- Dispense su Dependency Injection (Ultimo accesso: 02/03/2026)
- Dispense su OOP (Ultimo accesso: 02/03/2026)
- Dispense su Diagrammi delle classi (Ultimo accesso: 02/03/2026)
- Dispense su Pattern architetturali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern creazionali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern strutturali (Ultimo accesso: 02/03/2026)
- Dispense su Principi SOLID (Ultimo accesso: 02/03/2026)
- Dispense sulla Progettazione software (Ultimo accesso: 02/03/2026)
- Dispense sulla Qualità del software (Ultimo accesso: 02/03/2026)
- Repository Architettura esagonale (Kotlin) (Ultimo accesso: 02/03/2026)

- Repository Architettura esagonale (Java) (Ultimo accesso: 02/03/2026)
- Repository Ingegneria del software - swe-imdb (Ultimo accesso: 02/03/2026)
- C4 Model (Ultimo accesso: 03/03/2026)
- PlantUML (Ultimo accesso: 03/03/2026)
- Riferimenti a scelte tecnologiche:
  - TypeScript (Ultimo accesso: 03/03/2026)
  - Angular (Ultimo accesso: 03/03/2026)
  - Electron (Ultimo accesso: 03/03/2026)
  - SQLite (Ultimo accesso: 03/03/2026)
  - sqlite-vss (Ultimo accesso: 03/03/2026)
  - ONNX Runtime (Ultimo accesso: 03/03/2026)
  - Transformers.js (Ultimo accesso: 03/03/2026)
  - Node.js (Ultimo accesso: 03/03/2026)
- Glossario (v2.0.0)
- Piano di Progetto (v2.0.0)
- Piano di Qualifica (v2.0.0)
- Verbali interni:
- Verbali esterni:

2 Tecnologie utilizzate

La scelta delle teccnologie è stata guidata dalla necessità di rispetta i vincoli richiesti dalla proponente legati ad autoconsistenza, portabilità e facilità d'uso, nonché da considerazioni relative alla facilità di sviluppo, alla disponibilità di risorse e alla compatibilità con i requisiti funzionali e non funzionali del sistema.

3 Linguaggio di Programmazione: TypeScript

Scelta	Versione	Motivazione
TypeScript	5.9.2	Scelto per la sua capacità di fornire un sistema di tipi statico che migliora la sicurezza del codice e facilita la manutenzione, riducendo il rischio di errori comuni durante lo sviluppo. Inoltre, TypeScript è ampiamente supportato da un ecosistema ricco di librerie e strumenti, e si integra perfettamente con tecnologie come Angular ed Electron.

Tabella 2: Dettagli sulla scelta di TypeScript come linguaggio di programmazione

3.1 Deployment: Electron

Scelta	Versione	Motivazione
Electron	30.0.0	Scelto per la sua capacità di consentire lo sviluppo di applicazioni desktop multipiattaforma utilizzando tecnologie web, garantendo così la portabilità del prodotto su diversi sistemi operativi. Electron permette di creare un'esperienza utente nativa, sfruttando le potenzialità di Node.js per l'accesso al filesystem e alle risorse locali, facilitando l'integrazione con librerie e strumenti di terze parti.

Tabella 4: Dettagli sulla scelta di Electron come piattaforma di deployment

3.2 Framework Frontend: Angular

Scelta	Versione	Motivazione
Angular	21.2.5	Scelto per la sua capacità di supportare lo sviluppo di applicazioni web complesse e scalabili, con un'architettura basata su componenti che favorisce la modularità e la manutenibilità del codice. Inoltre, Angular offre un'ampia gamma di strumenti e funzionalità integrate, come la gestione dello stato, il routing e il supporto per la creazione di interfacce utente reattive, che lo rendono adatto a soddisfare i requisiti funzionali del progetto. La sua implementazione della Dependency Injection facilita la gestione delle dipendenze tra i componenti, migliorando la testabilità e la manutenibilità del codice.

Tabella 6: Dettagli sulla scelta di Angular come framework frontend

3.3 Librerie Aggiuntive

Scelta	Versione	Motivazione
Better SQLite	12.8.0	SQLite è un database relazionale che non richiede un server separato e può essere integrato direttamente nell'applicazione, facilitando lo sviluppo e il deployment. La sua leggerezza e portabilità lo rendono adatto a un'ampia gamma di piattaforme, garantendo la compatibilità con i requisiti di portabilità del progetto.
sqlite-vss	0.1.2	Scelto per la sua capacità di fornire una versione virtuale di SQLite, che consente di eseguire query su dati in memoria senza la necessità di creare un database fisico dei vettori per la ricerca semantica, semplificando così l'integrazione con il sistema.
ONNX Runtime	1.17.3	Scelto per la sua capacità di eseguire modelli di machine learning in modo efficiente e flessibile, supportando una vasta gamma di modelli e framework, facilitando l'integrazione di funzionalità di intelligenza artificiale nel prodotto.
Transformers.js	3.8.1	Scelto per la sua capacità di fornire un'interfaccia JavaScript per l'utilizzo di modelli di trasformatori, facilitando l'integrazione di funzionalità avanzate di elaborazione del linguaggio naturale e machine learning nel prodotto.

Tabella 8: Dettagli sulle scelte di librerie aggiuntive

3.4 Tecnologie di analisi statica

Scelta	Versione	Motivazione
SonarQube	2026.2	Scelto per la sua capacità di identificare e segnalare pattern di codice problematici, migliorando la qualità del codice e facilitando la manutenzione del progetto. Integrato nella pipeline di sviluppo, SonarQube consente di monitorare costantemente la qualità del codice, identificando potenziali bug, vulnerabilità e problemi di manutenzione, contribuendo così a garantire un prodotto finale robusto e affidabile.
Prettier	2.9.3	Scelto per la sua capacità di formattare automaticamente il codice in modo coerente, migliorando la leggibilità e facilitando la collaborazione tra i membri del team. Prettier supporta una vasta gamma di linguaggi e può essere facilmente integrato con editor di codice e strumenti di sviluppo, semplificando il processo di mantenimento di uno stile di codifica uniforme.

Tabella 10: Dettagli sulle scelte di tecnologie di analisi statica

3.5 Tecnologie di analisi dinamica

Scelta	Versione	Motivazione
Vitest	2.3.1	Scelto per la sua capacità di fornire un ambiente di test per applicazioni JavaScript e per il suo controllo dei componenti di Mock.
Playwright	1.59.1	Scelto per la sua capacità di fornire un ambiente di test per applicazioni web, con possibilità di testare flussi completi di interazione utente. Utilizzato quindi per i test E2E.

Tabella 12: Dettagli sulle scelte di tecnologie di analisi dinamica

4 Contesto del sistema

I vincoli di progetto, come specificato nell'Analisi dei Requisiti, richiedono che il sistema sia portabile, facile da usare e autoconsistente. Il caso d'uso principale del sistema è quello di un utente che desidera esplorare e interrogare un DIP per estrarre documenti fiscali e non, potendone verificare la validità e autenticità.

4.1 Modello architetturale

I vincoli di progetto hanno guidato la progettazione del sistema, influenzando le scelte tecnologiche e architetturali. Il sistema progettato quindi è pensato per non utilizzare tecnologie che richiedano installazioni o configurazioni complesse da parte dell'utente. I servizi offerti sono tutti accessibili in assenza di connettività, complesa la ricerca semantica.
Gli unici attori esterni coinvolti sono il file *system locale* da cui viene letto il DIP. STAMPA DA VEDERE SE HA SENSO METTERLO

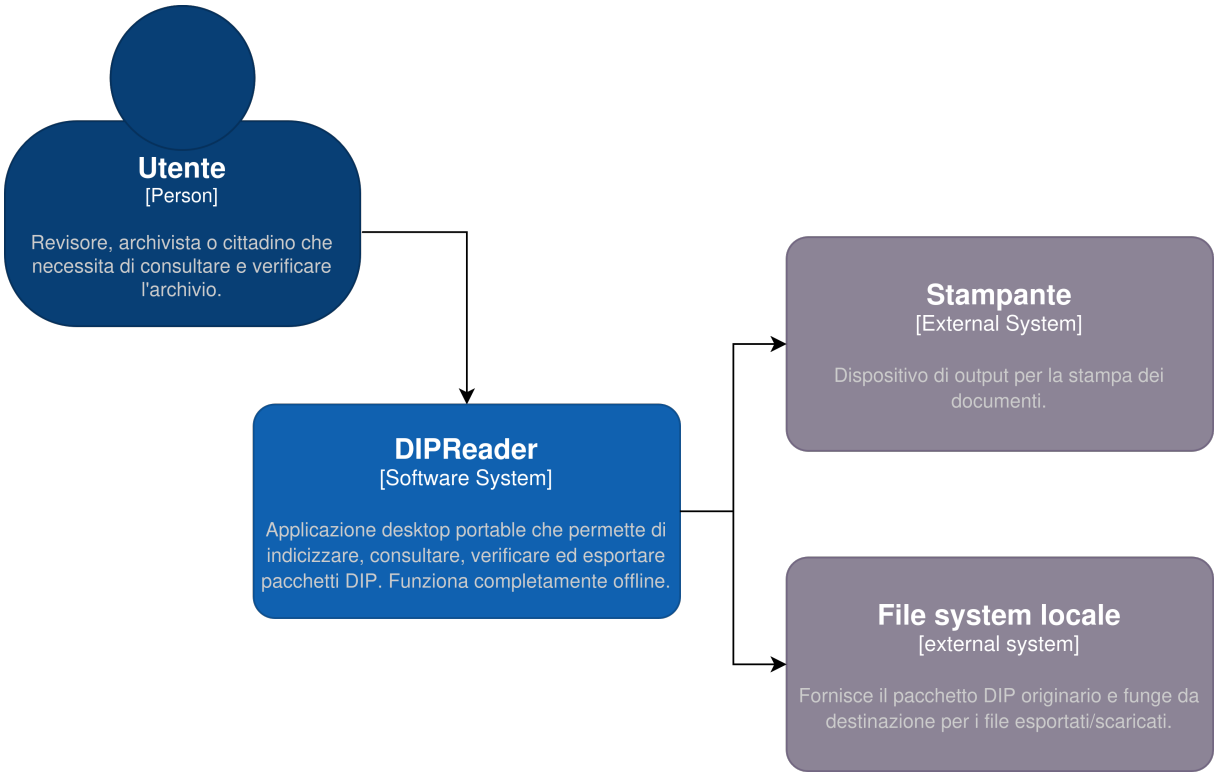

Figura 1: Diagramma C1 - Contesto del sistema *DIPReader*

4.2 Descrizione degli attori esterni

Gli unici attori esterni come già detto sono:

- il file system locale da cui viene letto il DIP
- STAMPA DA VEDERE SE HA SENSO METTERLO

5 Architettura dei container

La progettazione del sistema *DIPReader* è stata guidata dai principi di modularità, scalabilità e manutenibilità, con l'obiettivo di creare un'architettura che possa evolversi nel tempo e adattarsi a nuove esigenze e normative AGID.
In particolare, è stata completamente scorporata la logica di business dalla logica di presentazione, esponendo come unico punto di accesso comune gli *IPC channel*.
Il sistema così facendo permette di avere un backend facilmente estendibile, modificabile e soprattutto che si adatta allla struttura del DIP autonomamente. D'altra parte il frontend è completamente disaccoppiato ma strutturato in modo che i componenti di presentazione siano facilmente sostituibili, modificabili e soprattutto adattabili a nuove esigenze.

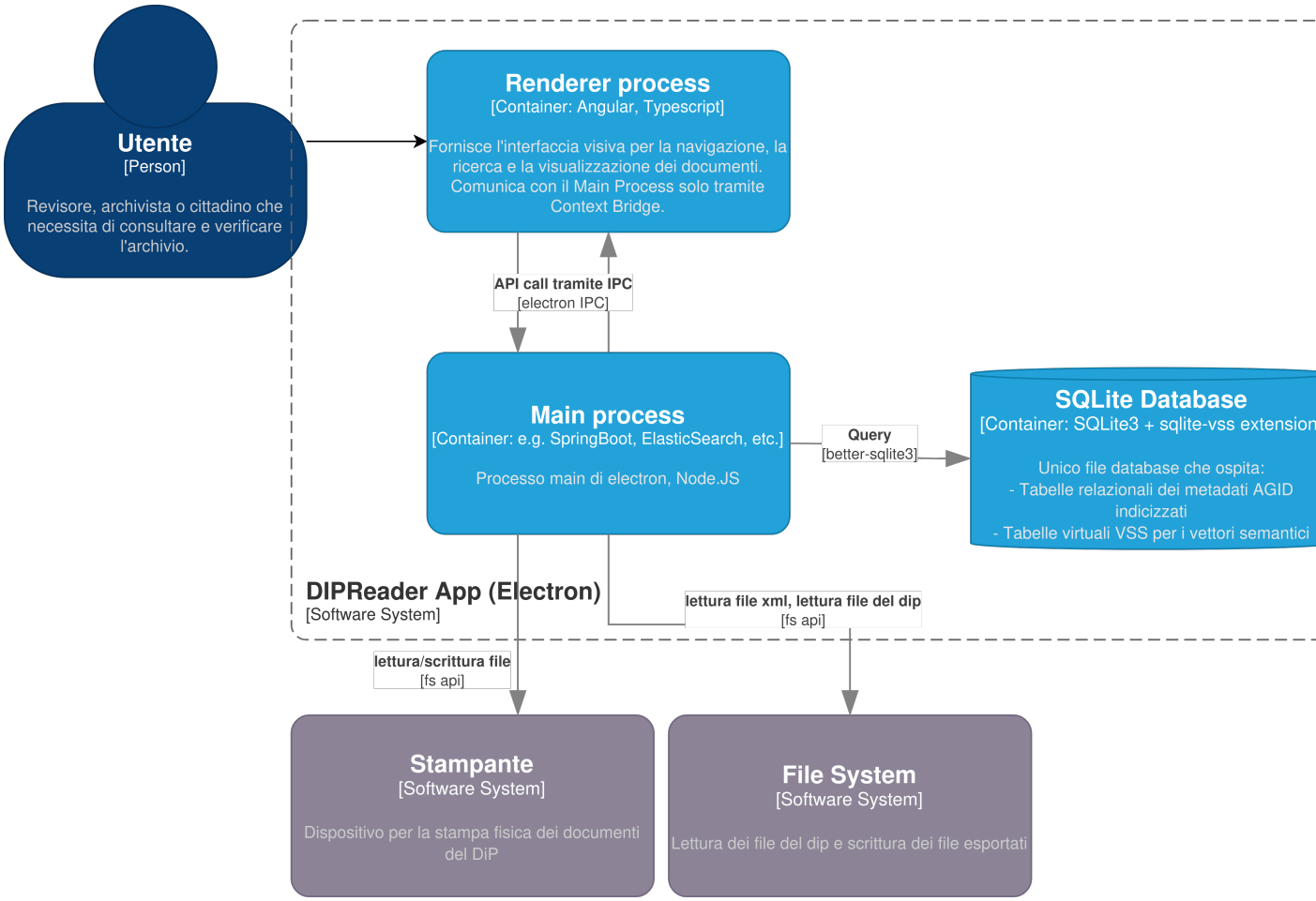

Figura 2: Diagramma C2 - Container del sistema *DIPReader*

5.1 Renderer

5.1.1 Struttura architetturale

Il container *Renderer* è responsabile della presentazione dell'interfaccia utente e dell'interazione con l'utente. Per la gestione quindi dell'interfaccia è stato scelto di strutturare il container per componenti. Ogni componente a sua volta è strutturato da una serie di services che si occupano di gestire la logica di presentazione, e da una serie di componenti di presentazione che si occupano di mostrare i dati all'utente.
Visto poi che le interazioni tra i diversi componenti di presentazione sono limitate, è stato scelto di strutturare il container in modo che ogni componente sia completamente disaccoppiato dagli altri. Gli unici punti di contatto risultano infatti essere il sistema di routing e i canali IPC, che permettono ai componenti di comunicare tra loro e con il backend.
In presenza poi di componenti come per esempio l'esportazione dei file, che richiedono di essere inseriti in più punti del sistema, è stato scelto di strutturare il container in modo che questi componenti siano facilmente integrabili in qualsiasi punto del sistema, senza dover modificare la struttura del container o dei componenti di presentazione grazie a delle semplici Injection di dipendenze.
È poi stato applicato il pattern Smart and Dumb Components, che permette di separare la logica di presentazione pura dalla logica di business che la coordina.

5.2 [Main]

5.2.1 Tecnologia: Node.js

Descrizione

6 Architettura delle componenti

6.1 [Componente 1]

6.1.1 Descrizione

6.1.2 Interfacce

6.1.3 Dipendenze

6.1.4 Tracciamento dei requisiti

6.2 [Componente 2]

6.2.1 Descrizione

6.2.2 Interfacce

6.2.3 Dipendenze

6.2.4 Tracciamento dei requisiti

6.3 Componenti di Verifica Integrità (Frontend)

6.3.1 Descrizione

La funzionalità di verifica integrità garantisce la correttezza e l'autenticità dei pacchetti DIP, verificando ricorsivamente l'integrità crittografica (generalmente hash SHA-256) dei file contenuti, propagando poi il risultato alla gerarchia documentale.
Per gestire l'albero delle dipendenze documentali sono stati adottati diversi design patterns:

- **Facade Pattern**: tutta la logica dello state management è isolata in *integrity.facade.ts*. In questo modo la facade agisce come single-source-of-truth per tutti i componenti di UI, inoltre coordina le chiamate ai servizi e gestisce lo stato interno *isVerifying* e di invalidazione della cache quando necessario.
- **Angular Signals**: Angular contribuisce con il suo sistema di state management reattivo basato su Signals, che permette di aggiornare automaticamente la UI in risposta ai cambiamenti di stato senza dover programmarne manualmente l'implementazione e limita l'aggiornamento solo ai componenti interessati, migliorando le performance.
- **Gateway Pattern**: come per le altre funzionalità, anche per la verifica integrità è stato adottato il Gateway Pattern per isolare la logica di comunicazione con il backend attraverso *integrity-ipc-gateway.ts*.
- **Smart/Dumb Components**: i componenti di presentazione (dumb) sono responsabili solo della visualizzazione dei dati e dell'emissione di eventi, mentre i componenti smart gestiscono la logica di business e le interazioni con la facade. In questo modo si ottiene una separazione chiara delle responsabilità e una maggiore riusabilità dei componenti di presentazione.

6.3.2 Componente di verifica

Integrity-Dashboard Il componente di presentazione è rappresentato da *integrity-dashboard.component.ts*, che si occupa di visualizzare lo stato della verifica integrità dell'intero DIP e di emettere eventi per avviare la verifica. Attraverso il metodo *loadOverview()* vengono recuperati i dati necessari per visualizzare lo stato del pacchetto, mentre il metodo *startVerification()* avvia la procedura di verifica integrità, delegando alla facade la gestione dello stato e delle chiamate al backend. Mentre la verifica è in corso la proprietà *_isVerifying.set(true)* permette di mostrare un indicatore di caricamento e disabilitare le interazioni con la UI fino al termine del processo. Al termine della verifica, la facade svuota la cache del renderer per poter aggiornare lo stato di verifica dei nodi dell'albero e dei componenti di dettaglio che includono informazioni di integrità.

6.3.3 Componenti di presentazione

Integrity-Summary-Cards Questo *dumb component* include tre card che visualizzano il numero di processi verificati validi, non validi e ancora da verificare.

Integrity-Valid-Panel Questo *dumb component* raccoglie gli elementi verificati, validi e non validi, ponendo la visualizzazione di questi ultimi in cima alla lista. La logica di presentazione degli elementi varia in base al tipo di nodo e allo stato di verifica dei nodi di pari livello, ad esempio se un documento è stato verificato valido la lista lo includerà e rappresenterà come documento, mentre se oltre ad esso sono stati verificati tutti gli altri documenti di pari livello (ovvero dello stesso processo) e sono risultati tutti validi, allora il componente mostrerà un riepilogo del processo invece che dei singoli documenti. In questo modo si ottiene una visualizzazione più compatta e intuitiva dello stato di verifica integrità dell'intero pacchetto. Per i nodi non validi la lista non si limiterà a etichettarli in base al loro livello nell'alberatura documentale, ma mostrerà l'intero percorso del nodo a partire dal livello più alto (cioè la classe documentale) fino al nodo stesso in stile "matrioska", in modo da evidenziare immediatamente la posizione del nodo non valido.

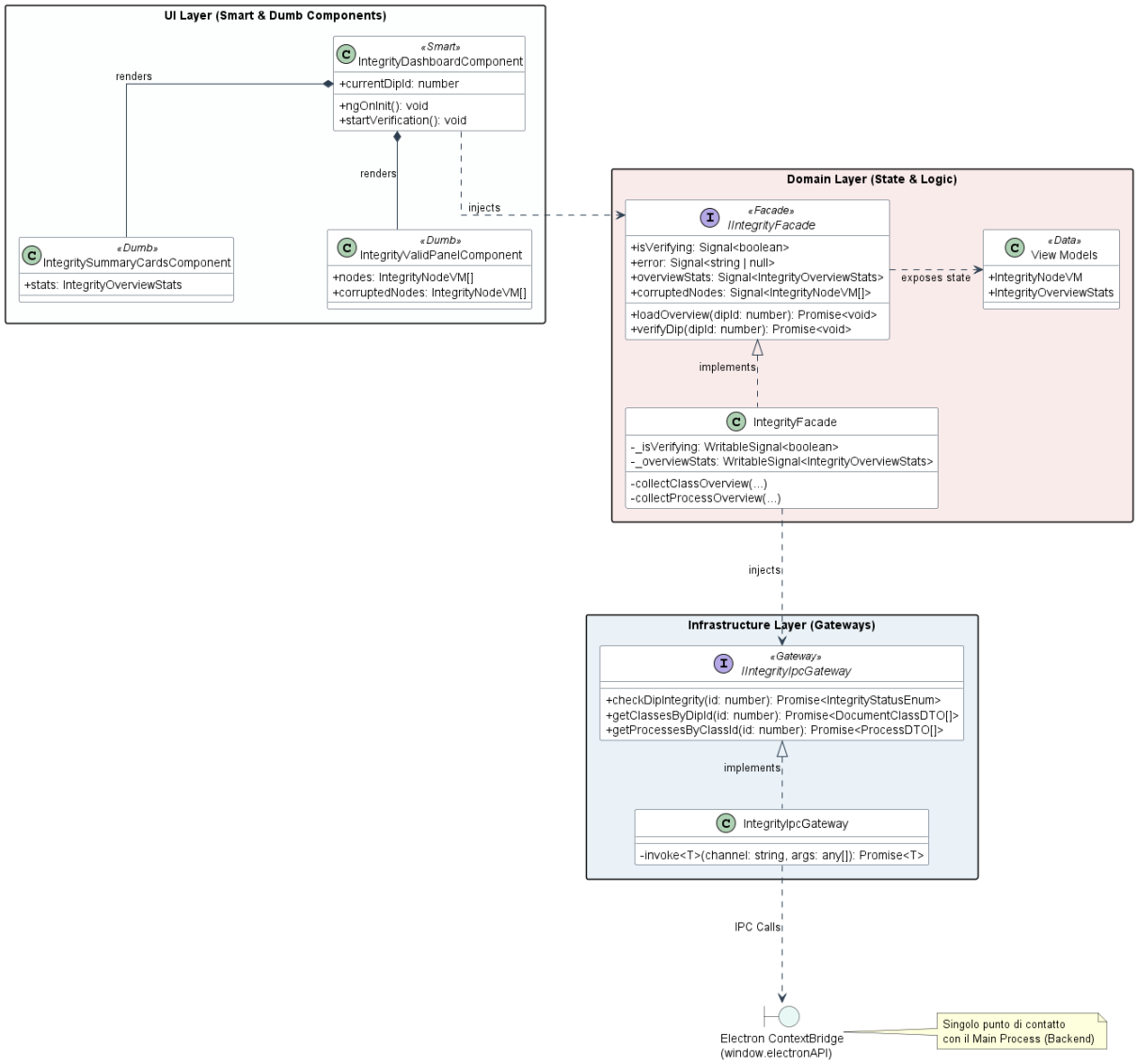

Figura 3: Diagramma delle classi Verifica Integrità (Frontend)

6.4 Componenti di Ricerca (FrontEnd)

6.4.1 Descrizione

I componenti di ricerca sono responsabili della gestione delle funzionalità di compilazione dei campi di ricerca.
Come già anticipato il pattern applicato è quello di Smart and Dumb Components, per cui i componenti di ricerca sono suddivisi in componenti di pura presentazione e gestiti da i componenti "*Padri*", che si occupano di gestire la logica di questi ultimi nonchè le interazioni con altri servizi come la validazione dei filtri, la gestione dei campi di ricerca dinamici e il routing della sezione dei risultati. Nel contesto della ricerca questo coomponente Padre è SearchPageComponent.
Come si può notare questo componente sfrutta la dependency injection nativa di Angular per risolvere le dipendenze. Inoltre sono iniettate le interfaccie dei servizi necessari

per il funzionamento e non gli effettivi servizi, applicando il principio di sostituzione di Liskov. La page si appoggia poi alla SearchFacade per gestire tutte le interazioni con i servizi, nascondendo la complessità di questi ultimi e fornendo un'interfaccia semplificata per la gestione della logica di ricerca.

6.4.2 Componenti di presentazione

I componenti di presentazione pura sono costruiti in modo da adattarsi completamente alle specifiche descritte dall'Allegato 5 dell'AGID e dalle linee guida di Sanmarco Informatica.
Posso essere suddivisi in tre categorie principali:

- **Componenti di ricerca**: sono i componenti che gestiscono la compilazione dei filtri di ricerca.
- **Componenti di visualizzazione**: sono i componenti che gestiscono la visualizzazione dei risultati di ricerca.
- **Search bar**: è il componente che si occupa della ricerca di processi, classi documentali e ricerca semantica.

Componenti di ricerca

AdvancedFilterPanelComponent I diversi filtri di ricerca, basati sui metadati AGID, sono raggruppati nel *AdvancedFilterPanelComponent*. Anche in questo caso le dipendenze sono risolte grazie alla dependency injection di Angular. La responsabilità di questo componente è quella di gestire la compilazione dei filtri fi ricerca, delegando le logiche secondarie ai servizi iniettati. In particolare, è iniettata l'interfaccia del servizio di validazione dei filtri, che si occupa di validare i filtri di ricerca dinamici, e il servizio di gestione dei campi di ricerca dinamici, che si occupa di gestire i campi di ricerca dinamici.

FilterRulesManager La gestione dei campi di ricerca risulta infatti fondamentale per garantire che la query di ricerca sia sensata. Dal momento che i campi sono differenti a seconda del tipo di documento, è obbligatorio forzare l'utente a compilare solo i campi di ricerca pertinenti al tipo di documento. Questo compito è delegato al *FilterRulesManager*.

Filtri semplici La maggior parte dei filtri di ricerca sono semplici campi di testo o dropdown, che non richiedono una logica complessa per la loro gestione. Segue quindi un elenco di questi componenti di UI pura, al fine di mostrare la completa aderenza alle specifiche AGID e alle linee guida di Sanmarco Informatica.

- *CommonFiltersComponent*: è il componente che gestisce i filtri comuni a tutti i tipi di documento (Anni di Conservazione, Chiave descrittiva, Classificazione con i corrispettivi sottocampi)

- *DiDaiFiltersComponent*: è il componente che gestisce i filtri specifici per i documenti informatici e amministrativi informatici (tipologia di documento, modalità di formazione, Nome, versione id del documento, informazioni sull'identificativo di formato, dati di veriffica e tracciatura con riguardo alla tracciatura delle modifiche)
- *AggregateFiltersComponent*: è il componente che gestisce i filtri specifici per le aggregazioni documentali (tipologia e id di aggregazione, data di apertura e chiusura, dati di procedimento e delle sue fasi e dati sulle assegnazioni)

Filtri dei soggetti I filtri dei soggetti sono invece più complessi. La loro gestione è realizzata secondo un approccio a strategie, così da rendere il comportamento dell'interfaccia dipendente dal contesto documentale selezionato e non da logiche condizionali distribuite nel codice. Il sistema consente di abilitare, disabilitare e specializzare i criteri disponibili in funzione del tipo di documento, mantenendo separati gli aspetti di presentazione, configurazione delle regole e costruzione della query.
L'interazione dell'utente avviene attraverso il pannello dei filtri avanzati, che funge da contenitore dello stato complessivo e coordina i diversi sotto-filtri. All'interno di tale pannello, il componente dedicato ai soggetti gestisce un wizard guidato: in base al DocContext viene selezionata, tramite una registry di strategie, la configurazione dei ruoli consentiti e dei relativi tipi di soggetto. Una seconda registry definisce invece i campi di dettaglio associati a ciascun SubjectType, consentendo la generazione dinamica del form di compilazione.
I criteri inseriti vengono raccolti come *SubjectCriteria* e successivamente integrati nello stato complessivo dei filtri. In fase di esecuzione della ricerca, tali valori vengono trasformati in una struttura di query gerarchica compatibile con il back-end, così da mantenere una separazione netta tra modello di interfaccia e modello di ricerca. Questo consente di garantire coerenza funzionale, facilità di estensione e una più chiara manutenzione delle regole di validazione e composizione dei filtri.

Filtri Custom I filtri custom sono metadati personalizzati di un documento costituiti da una chiave e un valore. Per soddisfare la richiesta dell"azienda proponente di esporre all'utente una lista di chiavi che rappresenta quelle effettivamente presenti nei documenti, è stato realizzato un componente

Componenti di visualizzazione La visualizzazione dei risultati di ricerca deve rendere possibile la visualizzazione all'utente di anteprime di processi, classi, aggregazioni e documenti. Queste visualizzazioni sono naturalmente diverse. Per esempio solo i documenti possono essere stampati o scaricati. Le informazioni mostrate per i processi sono invece differenti da quelle mostrate per i documenti e dalle classi.
La necessità è quindi quella di mostrare in modo dinamico i risultati di ricerca in base al tipo di risultato che viene ritornato. Per questo motivo è stato scelto di sfruttare un pattern *Factory*.
il *SearchResultFactory* si occupa di istanziare dinamicamente il componente di visualiz-

zazione corretto in base al tipo di risultato di ricerca sfruttando un registry che associa al tipo di risultato il componente di visualizzazione corrispondente .

Search bar La Search bar è invece gestita da un componente dedicato. La sua responsabilità è quella di permettere all'utente di ricercare processi tramite id di processo, classi per nome e ricerche di testo libero sfruttando la ricerca semantica.
Oltre al compoenente di presentazione pure è stato realizzato un servizio dedicato alla logica del componente. Per la ricerca semantica infatti è stato aggiunto un checkbox. Alla versione attuale del codice la ricerca semantica è abilitata per la ricerca di documenti ma se in un futuro si volesse estendere questa funzionalità basterebbe modificare il servizio *SearchBarRulesManager*